

LOGICTAP TAKE-HOME ASSIGNMENT

Candidate: Bandharu Mounika Lakshmi Jyothirmayi | Date: 6 October 2026

PART 1 - DEBUGGING

1. Why the patient was called again

- At 10:02:19 the call was answered and a 45-second timer started.
- The caller kept speaking, but nothing cancelled the timer.
- At 10:03:04 it fired mid-call, marked the job *failed*, set `attempts=1` and queued a retry for +30 minutes.
- The call then ended *answered*, so the job became *completed*. The queued retry was never cancelled.
- A duplicate webhook (40 ms later) pushed attempts to 3.
- At 10:33:04 the retry ran. `run_job` does not check state, so it dialled a completed job.

2. Problems, in the order I would fix them

- `run_job` dials without checking state or attempts.
- The no-response timer is not cancelled on speech or call end (stale timer).
- No state rules: *failed* and *completed* overwrite each other, and a finished job's retry is not cancelled.
- Duplicate webhooks are not deduplicated (no event ID), which inflates attempts.
- Attempts are counted in two handlers instead of once per dial.
- Read-modify-write with no transaction, so the timer and a webhook can race.
- The phone number is logged in plain text.

3. Fix for problem 1

Claim the job atomically before dialling. Retry scheduling sets the state back to *queued*.

```
UPDATE jobs SET state='dialling', attempts=attempts+1
WHERE id=? AND state='queued' AND attempts<max_attempts
```

If zero rows change, log "skipped" and return without dialling. Two workers cannot both claim the job, and a completed job or one at max attempts is never dialled.

PART 2 - WEBHOOKS

1. Tool used

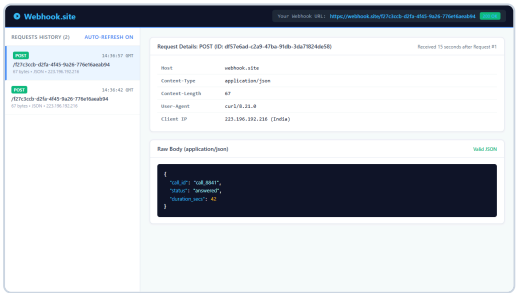
webhook.site (Unique URL: `https://webhook.site/f27c3ccb-d2fa-4f45-9a26-776e16aeab94`)

2. Command

```
curl -i -X POST https://webhook.site/f27c3ccb-d2fa-4f45-9a26-776e16aeab94 \
-H "Content-Type: application/json" \
-d '{"call_id": "call_8841", "status": "answered", "duration_secs": 42}'
```

(Executed twice from PowerShell terminal)

3. Screenshot



4. How a duplicate request could be detected

Looking strictly at the tool: both requests show the exact same JSON body (`{ "call_id": "call_8841", "status": "answered", "duration_secs": 42 }`), same HTTP method (`POST`), same IP (`223.196.192.216`), and identical Content-Length (67 bytes). They only differ by arrival timestamp (14:36:42 vs 14:36:57 GMT) and the tool's internal request UUID. A server seeing identical body fields for the same `call_id` knows the second request is a duplicate.

5. What I would add to the request

An **Idempotency-Key** or **X-Event-ID** header (e.g. `Idempotency-Key: evt_8841_call_ended`) and `X-Delivery-Attempt: 2` , allowing the server to deduplicate requests via an O(1) cache lookup before even parsing the body.

6. What I learned

Duplicate webhooks carry identical business payloads but distinct transport-level metadata (different TCP ports and server-assigned request IDs), so deduplication must happen at the application layer. Also, Windows PowerShell strips raw double quotes from JSON arguments passed to external CLI tools like `curl` unless single-quoted or escaped.

PART 3 - SMALL SERVICE

1. Version 1

First untouched AI-generated code: [version1.py \(GitHub\)](#)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional

app = FastAPI()
calls_db = {} # In-memory dict

class CallEndedPayload(BaseModel):
    call_id: Optional[str] = None
    status: str
    duration_secs: int

@app.post("/call-ended")
def call_ended(payload: CallEndedPayload):
    if not payload.call_id:
        raise HTTPException(status_code=400, detail="Missing call_id")
    if payload.call_id in calls_db:
        return {"status": "success", "message": "Call already recorded", "data": calls_db[payload.call_id]}
    record = payload.dict()
    calls_db[payload.call_id] = record
    return {"status": "success", "message": "Call recorded", "data": record}

@app.get("/calls/{call_id}")
def get_call(call_id: str):
    if call_id not in calls_db:
        raise HTTPException(status_code=404, detail="Call not found")
    return calls_db[call_id]
```

2. Final version / GitHub link

Full production implementation & tests: <https://github.com/Mounika4406/logictap-call-service>

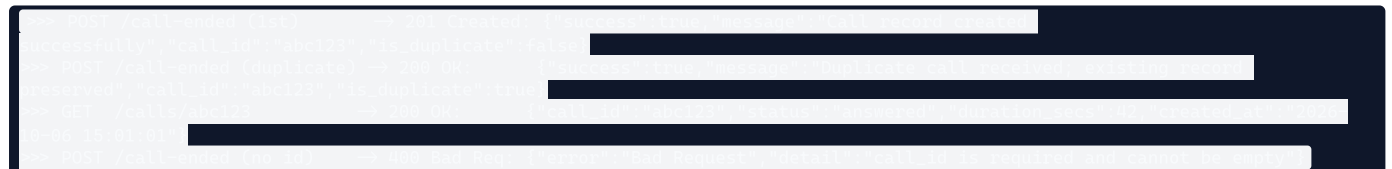
3. AI prompts

- "Write a Python FastAPI service with two endpoints: `POST /call-ended`: accepts JSON like `{'call_id': 'abc123', 'status': 'answered', 'duration_secs': 42}` and stores it. If the `call_id` already exists, don't store a duplicate and return success. If `call_id` is missing, reject it with an error. `GET /calls/{call_id}`: returns the stored call or 404 if not found."
- "The in-memory dict has concurrency race conditions under duplicate webhook arrival and loses data on reboot. Refactor this to use SQLite with atomic idempotency using `INSERT OR IGNORE` and a `PRIMARY KEY`. Add Pydantic v2 validators rejecting empty/whitespace `call_id` and negative duration, return HTTP 201 for fresh creations vs 200 for duplicates, and map missing `call_id` to HTTP 400 Bad Request."

4. What changed (2-3 lines)

- Replaced ephemeral in-memory dict with SQLite persistent storage using `PRIMARY KEY (call_id)` and atomic `INSERT OR IGNORE` to eliminate race conditions under concurrent webhook arrivals.
- Upgraded to Pydantic v2 validation enforcing non-blank `call_id` and non-negative `duration_secs`, returning structured HTTP 400 Bad Request errors.
- Standardized HTTP status codes: returning HTTP 201 Created on fresh inserts and HTTP 200 OK on idempotent duplicate replays.

5. Proof it runs



6. Automated test

Why it matters: It verifies end-to-end idempotency by posting the same payload twice, asserting both calls succeed, and confirming the database record count remains exactly 1.

```
def test_repeated_request_does_not_create_second_record():
    payload = {"call_id": "abc123", "status": "answered", "duration_secs": 42}
    resp1 = client.post("/call-ended", json=payload)
    assert resp1.status_code in [200, 201] and resp1.json()["success"] is True

    resp2 = client.post("/call-ended", json=payload)
    assert resp2.status_code == 200 and resp2.json()["success"] is True

    with sqlite3.connect(TEST_DB) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT COUNT(*) FROM calls WHERE call_id = ?", ("abc123",))
        assert cursor.fetchone()[0] == 1 # Exactly 1 record exists
```

Pytest Result: 4 passed in 0.78s

PART 4 - REFLECTION

1. AI mistake

The AI initially used an in-memory dictionary with a Python check (`if call_id in calls_db:`). In an async multi-worker setup, duplicate webhooks arriving milliseconds apart (like the two 40ms apart in Part 1) cause a race condition where both check before either writes, creating duplicates. I replaced this with an SQLite database using an atomic `PRIMARY KEY` and `INSERT OR IGNORE` query to enforce idempotency at the database engine level.

2. Tools used

AI: Gemini. **Runtime & Stack:** Python 3.14.7, FastAPI, SQLite3, Uvicorn, Pydantic v2. **Testing:** pytest, HTTPX. **Webhook Tools:** webhook.site, cURL, Google Chrome. **Version Control:** Git, GitHub.

3. Hardest part

Tracing how the uncanceled silence timer silently scheduled a 30-minute retry that later dialled without checking the completed job state.

4. Total time

48 minutes (Part 1: 15 min, Part 2: 12 min, Part 3: 18 min, Part 4: 3 min).